

Aegis

Guardrails an LLM agent cannot ignore

A deterministic, fail-closed policy gate for deploying LLM agents on production kdb+/q estates

Honest technical overview - engineering complete on the structured kdb+ analyst surface; remaining gates are human (real-data re-soak, design partner, third-party audit).

Working name. MIT licensed. Not a security guarantee or legal advice.

1 Problem statement

Banks and trading firms want LLM agents working **on and around production kdb+ systems** - tickerplants, real-time and historical databases, gateways. The value is real; so is the danger:

- q is a full programming language reached through the **same channel as a query**. One string can run shell, delete the HDB, kill the tickerplant (a market-data outage - a regulatory event), overwrite the sym file (silently corrupting every symbol column), open connections, or exfiltrate client positions and P&L.
- An LLM is non-deterministic and **promptable by its own inputs** - a prompt-injection in a tool result can make a well-aligned model act.

Why the obvious defenses fail. Anything in the model's prompt ("you must not delete data") is *advice* - attended to probabilistically and ignored some fraction of the time. Classifier / LLM-judge guardrails have been empirically evaded up to 100%. The field's conclusion: **never rely on the model for security-critical decisions.**

2 The core idea

A real control is a **deterministic checkpoint a separate process decides, before the action runs**. Aegis does not *ask* the model to behave; it *decides* whether each action happens.

- **Out-of-model, out-of-process.** The decision point runs where the agent can't tamper with it; the model's output is an *input*, never trusted.
- **Default-deny - enumerate goodness, not badness.** Only granted capabilities are allowed; blocking "bad" patterns on a Turing-complete language is a losing arms race.
- **Fail-closed.** Missing/forged policy, unreachable PDP, unknown tool, any error → block.
- **The query plane is bounded by construction.** The agent **never sends raw q** - it sends a *structured request* that Aegis **compiles** into bounded, allowlisted q. Dangerous operations have **no slot in the grammar**: structurally impossible, not merely detected.
- **Rows are governed, not just tables.** A mandatory, non-removable per-principal **row filter** is ANDed into every compiled query - so an analyst sees only the rows their entitlement allows, even through joins/set-ops or when they explicitly ask for rows outside their set.
- **Provenance decides privilege (the injection defense).** Every content item the agent sees is labelled trusted or untrusted; **untrusted content can never drive a privileged action** (a trade, an email, a write, an egress). The injection need not be detected - it simply cannot reach a trigger. This is information-flow control.
- **Confinement is load-bearing; the gate is defense-in-depth.** The kdb+ process runs non-root, read-only HDB, no shell, no egress, under a **seccomp syscall filter** - so even a gate bypass cannot destroy data, reach the network, or attack the kernel.

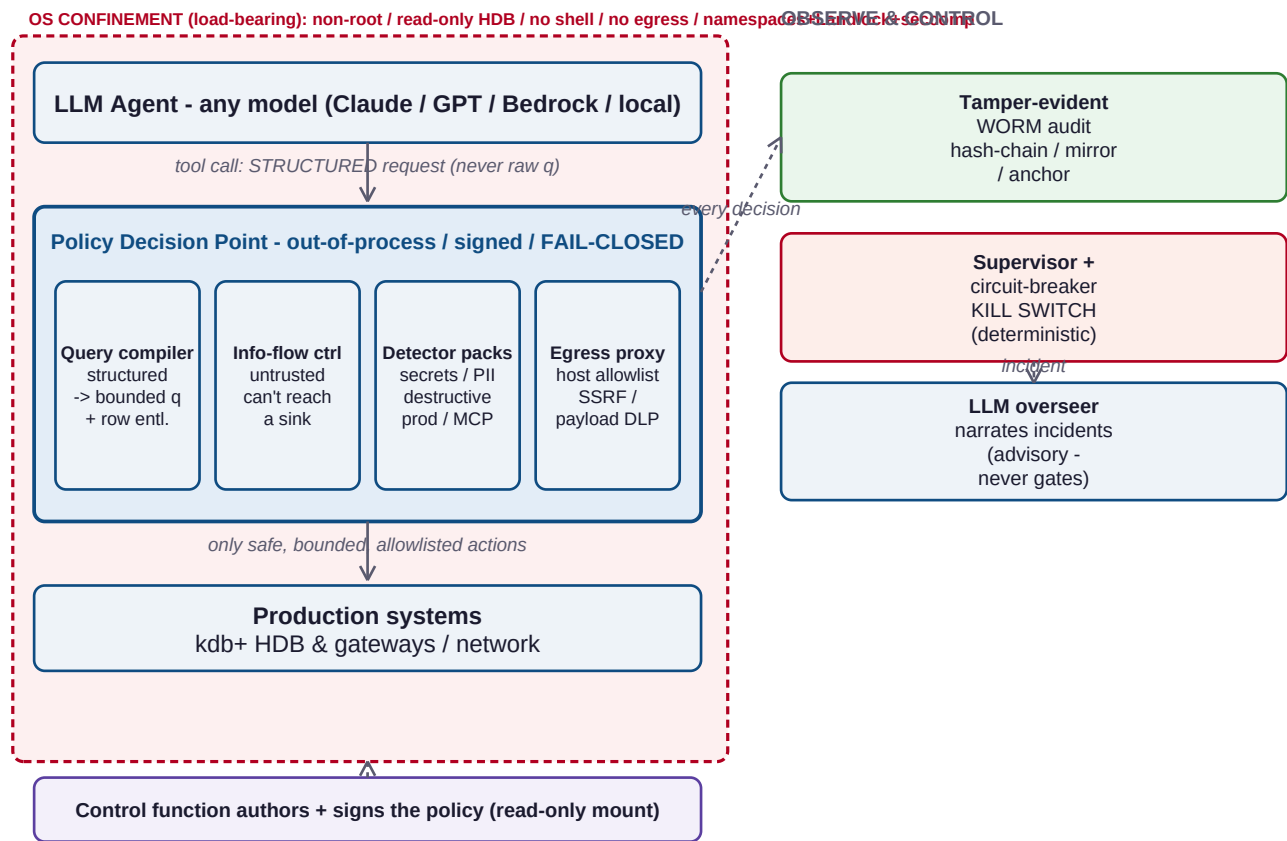
3 What it is tested to stop

The adversarial corpus - driven by an *uncooperative* model told to actually try - covers, on the kdb+ analyst surface:

1. Running OS / shell commands on the kdb+ host
2. Destroying or corrupting data on disk (HDB partitions, the sym file)
3. Mutating the live data (delete / insert / update)
4. Stealing sensitive / client data (positions, P&L, account_no, salary)
5. Reaching outside its lane (non-allowlisted tables, columns, or **rows** the principal is not entitled to)
6. Exfiltration & remote code (outbound connections, native shared-object load)
7. Hijacking the process itself (message-handler replacement, exit)
8. Resource exhaustion / DoS (unbounded scans that degrade the box)
9. Reading protected files (the policy, password lists, the audit log)

10. Evasion / injection (obfuscation, dynamic eval, injection via the API's own fields)
11. **Indirect prompt injection** - a malicious instruction hidden in a tool result (a file, a query's free-text field, a web/MCP response) that tries to drive a privileged action; stopped by information-flow control (untrusted-derived actions can't reach a sink)

4 Architecture



Component	Role	Property
PDP (engine, pdp_service)	decides every action against the signed policy	out-of-process, fail-closed
Query compiler	compiles structured requests to bounded q	injection structurally impossible
Row entitlements	mandatory per-principal row filter, ANDed into every table ref	non-removable, can't widen, covers joins
Information-flow control (ifc)	untrusted content can't reach a privileged sink	deterministic injection defense
Detector packs	veto: secrets / PII / destructive / prod / resource / MCP	deterministic, defense-in-depth
Egress proxy	network egress control	host allowlist + SSRF + payload DLP
Signing + change guard	Ed25519-signed policy; an edit may only narrow without approval	integrity; widening needs sign-off
Schema-drift linter	diffs the signed policy vs the live schema	catches lockout + unbounded-scan drift
WORM audit	tamper-evident record of every decision	hash-chain + off-host mirror + anchor
Supervisor + kill switch	trips a breaker + kills on behavioural tripwires	deterministic, load-bearing

LLM overseer	reads the audit, narrates incidents	advisory only, never gates, out-of-band
OS confinement	the containment boundary	namespaces + Landlock + seccomp

5 Customising it to your estate

The policy is **data**, owned by the control function - not code. Adapting Aegis to a specific desk or system is editing a signed JSON policy and running a validator: **no engineering, no rebuild of the engine.**

- **Schema, tables & columns.** The structured-query allowlist - allowed tables, per-table columns, required-date tables, row caps, permitted aggregations/operators - is declared in the policy. The validator (`aegis.policy_lint`) checks it is well-formed, and `aegis.policy_schema_diff` checks it against the **live schema** so a dropped column (silent lockout) or a new partitioned table without a date bound (unbounded scan) is caught before signing.
- **Row entitlements.** Per-principal row filters (which rows each principal may see - by region, book, symbol, desk) are declared in the policy and injected mandatorily into every compiled query; a wildcard baseline plus table-specific rules combine fail-safe.
- **Rules & threat packs.** Each pack (secrets, PII terms, destructive ops, prod markers, resource limits, MCP manifests, per-tool argument rules) is enabled and tuned in the policy. New deterministic rules ship as packs without touching the gate.
- **Information-flow policy.** Which tools are privileged sinks (egress, write, order, scoped query) and which columns/sources are untrusted (free-text fields, external tool results) are declared so the IFC veto knows what untrusted content must not drive.
- **Tool surface & principals.** Which named tools an agent (or a specific principal) may use is a grant list, with RBAC scoping per principal; the free-form/break-glass surface is a separate, separately-signed policy.
- **Supervisor, kill action & confinement.** Behavioural tripwires, the kill action (signal / docker kill / kubectl delete / callback), and the hardening profile (read-only mounts, dropped caps, egress allowlist, syscall filter, resource limits) are declarative and CI-checked against your real manifest.

The change workflow: author from the template, run the validators until clean, sign it (Ed25519), mount it read-only. Every edit is checked by the **policy-change guard**: a change that only narrows the allow-set auto-applies; one that **widens** it (grants any new capability) is refused until a human approves, and the exact new grant is named. Changing a rule = edit, re-validate, re-sign, reload the PDP - **no code change, fully audited**. A turnkey authoring kit lets the control function do all of this - and run the real-data re-soak - themselves.

6 How it works (request lifecycle)

- 1 The agent emits a tool call - on the analyst surface, a **structured query request** (data, not q text).
- 2 The PDP checks the **circuit breaker** (is this principal quarantined?), then **default-deny grants** (is the tool / table / column even allowed?).
- 3 The **query compiler** validates every field against allowlists and emits **bounded q** - a date filter is required on partitioned tables, the result is capped, the principal's **mandatory row-entitlement filter is ANDed in** (covering both sides of any join/set-op), and the output is re-checked against a backstop. Any off-allowlist field → reject.
- 4 The **information-flow veto** checks provenance: if the action's arguments were derived from **untrusted** content and the tool is a **privileged sink**, it is blocked before it runs - regardless of what the injected text said.
- 5 **Detector packs** and the **egress proxy** veto anything that passes.
- 6 The decision (allow / block / require-approval) is recorded to the **tamper-evident audit**, mirrored off-host.
- 7 The **supervisor** watches the decision *sequence*; behavioural tripwires trip a **circuit breaker** that quarantines the principal and fires a **kill switch**. The **LLM overseer** narrates the incident out-of-band (advisory; never delays a decision).
- 8 Only an allow reaches the real kdb+ gateway - and even then the agent runs inside **OS confinement** that physically prevents shell, file destruction, egress, and the dangerous syscalls.

7 How we tested it

Validated by deterministic, runnable proofs - not assertions.

- **Acceptance suite (CI-gated)**: 32 core batteries on Python 3.10-3.12, a wheel fresh-install smoke test, the deployment-hardening gate, and ruff - every push.
- **Formal**: the default-deny grant algebra proved sound & monotonic by exhaustion and by **Z3/SMT over unbounded domains**; AWS's open-source **Cedar Analysis CLI (CVC5)** independently corroborates the exported policy.
- **q-semantics conformance on real kdb+**: the compiler's emitted q is run against a live instance to prove the bounds hold - the materialisation cap holds, a reducing query is not corrupted, the entitlement predicate is effective, and the database is read-only afterwards.
- **OS confinement on real Linux**: the adversarial test and Landlock filesystem test (fail-closed), plus the **seccomp-bpf** syscall deny-list verified at the kernel level (a blocked syscall is killed with SIGSYS while benign ones run).
- **Production-scale realism soak (headline evidence)** on ~4 billion rows (4 FSP HDBs, 500M trade + 500M quote each, partitioned, 166 GB): **no hints** (schema discovered via meta); an **uncooperative attacker** (0 refused) → **0/30 attacks succeeded**; **ground-truth correctness** checked against independently-computed q. **Row entitlements** held at scale (an analyst sees only its entitled rows, even through joins or when asking for others). The process found & fixed real defects in the primary control - exactly what scale + ground-truth checking exist to catch.
- **Prompt injection (information-flow control)**: on the estate, a real indirect-injection corpus gave **100% block, 0 benign false positives**. On the **AgentDojo** standard benchmark (official, Opus) the agent keeps real utility and targeted-attack-success sits at or near zero with IFC firing on the canonical injections; the clean with-versus-without delta on the egress suites is **partially complete** (an API-credit stop), documented and resumable - corroboration of an already-proven property, not a blocker.

8 Honest limitations - what Aegis is *not*

- It governs the agent's **actions**. It does not make the model truthful, doesn't stop it *suggesting* something unwise in text, and does not replace IAM / DLP / change-control - it composes with them.
- **The gate is not the containment.** A denylist over a Turing-complete language is leaky; the kernel confinement is what actually contains a determined agent.
- A namespace/Landlock/seccomp sandbox is **weaker than a microVM** against a kernel-level attacker (out of scope: a kernel 0-day in an allowed syscall, side channels). The seccomp layer blocks the clearly-dangerous syscalls, shrinking the largest part of that gap. For adversarial multi-tenant, run the same policy inside a microVM (Firecracker/Kata) - documented, not yet deployed.
- **The injection defense is structural, but its false-positive cost is surface-specific.** On the kdb+ analyst surface the false-positive rate is zero (query columns are structured trusted data). On a broad tool surface (chat, email, document agents), where benign actions routinely derive from untrusted content, the same rule can over-block and must be tuned per surface - the AgentDojo runs surface exactly this nuance.
- The reported numbers are on a **representative** corpus and a **synthetic** (if realistic-scale) schema, plus a partial run of the AgentDojo benchmark. They prove the design; they are **not** a production number. The control function must re-soak on the **real desk corpus and real data** before enforcing - the one gate only they can close.
- The free-form (raw-q) surface exists only as **admin-only break-glass**, separately signed, never granted to an analyst; it is honestly weaker than the structured surface.
- **Numeric overflow is the model's, not the gate's.** q sum over a 64-bit integer column wraps silently; the compiler bounds which rows are read and the result size, but does not widen aggregations. The conformance battery surfaces this; the fix is an estate-side schema choice (widen those columns to long).

9 Where this sits versus the frontier

A 2026 multi-source review places Aegis as strongly **aligned** with the deterministic, out-of-band enforcement direction the field has converged on, and **ahead** in one place: no competitor was found compiling agent queries as a safety gate (the closest research uses an LLM to *rewrite* queries - the opposite trust model), and none at the q/kdb+ plane. The information-flow layer brings the prompt-injection defense level with the research frontier. The honest residual gaps are a microVM substrate for fully-untrusted third-party code and completing the standard-benchmark delta.

10 Status & what remains

Engineering: complete and validated on the structured kdb+ analyst surface - bounded-by-construction query plane, mandatory row entitlements, deterministic prompt-injection defense, kernel confinement (namespaces + Landlock + seccomp), two-tier oversight + kill switch, signed out-of-process PDP with a policy-change guard, tamper-evident WORM audit, installable package, CI-gated at 32/32.

Remaining - the human gates (not code):

- Control-function **real-data re-soak** (the authoring kit makes this turnkey).
- A **design partner** running it in monitor mode on production traffic.
- A **third-party security audit** before a production estate depends on it.
- Completing the **AgentDojo official with-versus-without delta** on the egress suites (resumable; corroboration, not a blocker).

Recommendation: GO to enforce on the structured analyst surface, conditioned on confinement deployed (load-bearing), free-form kept off the analyst grant, and the signed PDP + WORM audit live.